

## DL Assignment 08

Name - KAVISH PARASWAR

Class - CS(AI) B

Batch - 01

Roll no - 02

PRN - 12311694

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
fashion_mnist = tf.keras.datasets.fashion_mnist

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 2s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 2s 0us/step
```

```
print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
```

```
Training shape: (60000, 28, 28)
Testing shape: (10000, 28, 28)
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
```

```
model = models.Sequential()

# Convolution Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D(2,2))

# Convolution Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D(2,2))

# Flatten
model.add(layers.Flatten())

# Dense Layer
model.add(layers.Dense(128, activation='relu'))

# Output Layer
model.add(layers.Dense(10, activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    X_train, y_train,
    epochs=9,
    validation_data=(X_test, y_test)
)
```

```
Epoch 1/9
1875/1875 ————— 42s 22ms/step - accuracy: 0.8703 - loss: 0.3599 - val_accuracy: 0.8739 - val_loss: 0.3409
Epoch 2/9
1875/1875 ————— 82s 22ms/step - accuracy: 0.8973 - loss: 0.2829 - val_accuracy: 0.8949 - val_loss: 0.2860
Epoch 3/9
1875/1875 ————— 38s 20ms/step - accuracy: 0.9096 - loss: 0.2439 - val_accuracy: 0.9044 - val_loss: 0.2632
Epoch 4/9
1875/1875 ————— 38s 20ms/step - accuracy: 0.9206 - loss: 0.2146 - val_accuracy: 0.9057 - val_loss: 0.2637
Epoch 5/9
1875/1875 ————— 37s 20ms/step - accuracy: 0.9286 - loss: 0.1899 - val_accuracy: 0.9107 - val_loss: 0.2501
Epoch 6/9
1875/1875 ————— 38s 20ms/step - accuracy: 0.9379 - loss: 0.1673 - val_accuracy: 0.9135 - val_loss: 0.2463
Epoch 7/9
1875/1875 ————— 40s 20ms/step - accuracy: 0.9439 - loss: 0.1488 - val_accuracy: 0.9138 - val_loss: 0.2582
Epoch 8/9
1875/1875 ————— 42s 22ms/step - accuracy: 0.9496 - loss: 0.1325 - val_accuracy: 0.9055 - val_loss: 0.2827
Epoch 9/9
1875/1875 ————— 39s 21ms/step - accuracy: 0.9568 - loss: 0.1158 - val_accuracy: 0.9057 - val_loss: 0.2934
```

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_test, y_test)
)
```

```
Epoch 1/10
1875/1875 ————— 39s 21ms/step - accuracy: 0.9610 - loss: 0.1046 - val_accuracy: 0.9134 - val_loss: 0.2910
Epoch 2/10
1875/1875 ————— 39s 21ms/step - accuracy: 0.9662 - loss: 0.0903 - val_accuracy: 0.9173 - val_loss: 0.3030
Epoch 3/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9690 - loss: 0.0804 - val_accuracy: 0.9144 - val_loss: 0.3222
Epoch 4/10
1875/1875 ————— 39s 19ms/step - accuracy: 0.9721 - loss: 0.0724 - val_accuracy: 0.9090 - val_loss: 0.3728
Epoch 5/10
1875/1875 ————— 37s 20ms/step - accuracy: 0.9763 - loss: 0.0624 - val_accuracy: 0.9085 - val_loss: 0.3523
Epoch 6/10
1875/1875 ————— 36s 19ms/step - accuracy: 0.9786 - loss: 0.0579 - val_accuracy: 0.9052 - val_loss: 0.4416
Epoch 7/10
1875/1875 ————— 43s 20ms/step - accuracy: 0.9802 - loss: 0.0514 - val_accuracy: 0.9113 - val_loss: 0.4026
Epoch 8/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9825 - loss: 0.0462 - val_accuracy: 0.9156 - val_loss: 0.4290
Epoch 9/10
1875/1875 ————— 39s 21ms/step - accuracy: 0.9837 - loss: 0.0446 - val_accuracy: 0.9079 - val_loss: 0.4734
Epoch 10/10
1875/1875 ————— 42s 21ms/step - accuracy: 0.9855 - loss: 0.0395 - val_accuracy: 0.9163 - val_loss: 0.4634
```

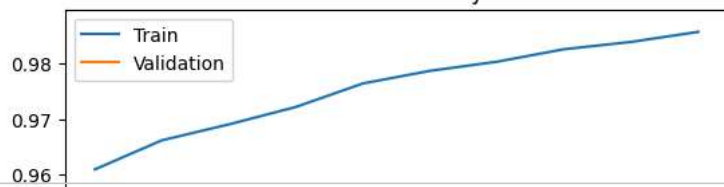
```
test_loss, test_acc = model.evaluate(X_test, y_test)
```

```
print("Test Accuracy:", test_acc)
```

```
313/313 ————— 3s 9ms/step - accuracy: 0.9163 - loss: 0.4634
Test Accuracy: 0.916299987602234
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['Train', 'Validation'])
plt.title("Model Accuracy")
plt.show()
```

Model Accuracy



```
predictions = model.predict(X_test)
```

```
print("Predicted Class:", np.argmax(predictions[0]))
```

```
print("Actual Class:", y_test[0])
```

```
319713 0.9713 2s 6ms/step
```

```
Predicted Class: 9
```

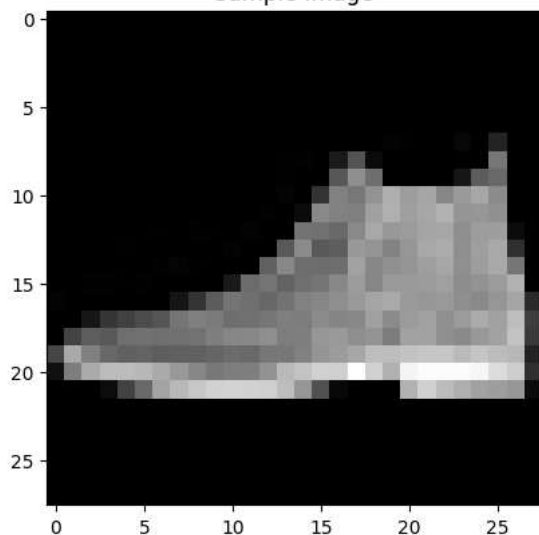
```
Actual Class: 9
```

```
plt.imshow(X_test[0].reshape(28,28), cmap='gray')
```

```
plt.title("Sample Image")
```

```
plt.show()
```

Sample Image



Start coding or [generate](#) with AI.